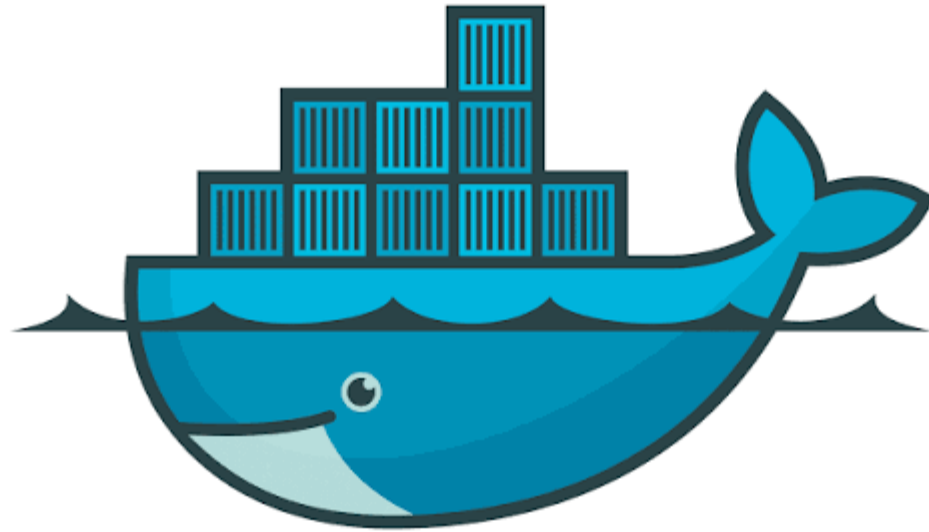


DevOps



Docker

Docker Port Mapping

Connect a port on the host to a port inside a container

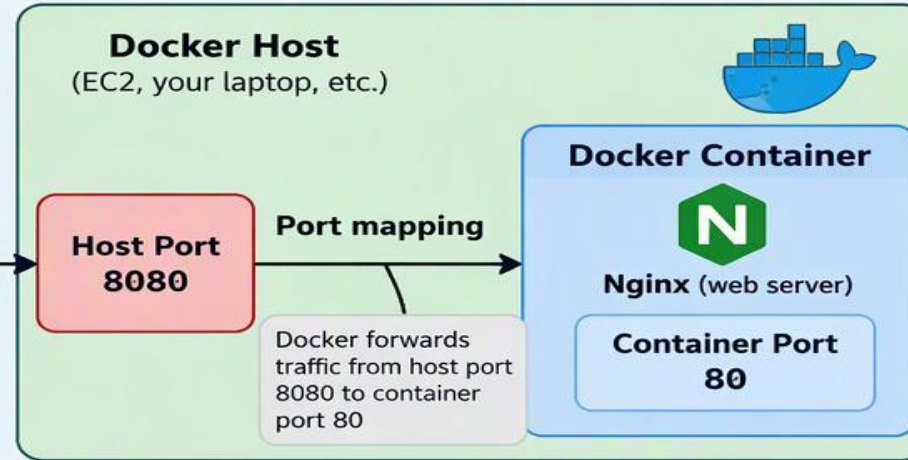
1. The Big Picture

Port mapping lets traffic from outside (your laptop, other machines, or the internet) reach an application running inside a Docker container.



Your Computer
(or Internet)

`http://<host-ip>:8080`
Request to host port 8080



2. The Command

Run a container and map host port 8080 to container port 80:

```
docker run -d -p 8080:80 nginx
```

Syntax:

```
docker run -p <HOST_PORT>:<CONTAINER_PORT> <image>
```

- Left side (8080) = port on the host
- Right side (80) = port inside the container
- You can use different port numbers

3. Example Access

After running the container, open in your browser:

```
http://localhost:8080
```

If Docker is on an EC2 server, use the server's public IP (and make sure the EC2 security group allows port 8080):

```
http://<EC2-PUBLIC-IP>:8080
```

4. `docker ps` Output

Check the port mapping with:

```
docker ps
```

You will see something like:

CONTAINER ID	IMAGE	PORTS
a1b2c3d4e5f6	nginx	0.0.0.0:8080->80/tcp

Host port (8080) Container port (80)

5. Multiple Containers

You can run multiple containers, each with a different host port, even if they use the same container port.

```
docker run -d -p 8081:80 nginx
docker run -d -p 8082:80 nginx
docker run -d -p 8083:80 nginx
```

Access them at:

- `http://localhost:8081`
- `http://localhost:8082`
- `http://localhost:8083`

6. Important Notes

- ✓ Containers have their own private IP addresses (e.g., 172.17.0.2).
- ✓ You can access a container directly from the Docker host or other containers on the same Docker network.
- ✓ To access from your computer or the internet, you need port mapping.
- ✓ The EXPOSE instruction in a Dockerfile only documents the intended port — it does not publish the port.
- ✓ Always allow the corresponding host port (e.g., 8080) in your firewall/security group if accessing remotely.



Key Takeaway

`-p HOST:CONTAINER`

maps a port on the Docker host to a port inside the container, so external users can reach your application.



Docker ENTRYPOINT vs CMD

Both define what runs in a container, but they serve different purposes.

ENTRYPOINT

“The fixed executable”

- Defines the **main command** (executable) that always runs when the container starts.
- Usually set by the image author.
- Does not get overridden easily (only with `--entrypoint`).
- You can still pass additional arguments at runtime (which are appended).

Dockerfile

```
ENTRYPOINT ["/usr/bin/myapp"]
```

This sets `/usr/bin/myapp` as the command that will always run.

CMD

“The default arguments”

- Provides default arguments to the command (often used with ENTRYPOINT).
- Can be easily overridden by providing a command/arguments when running the container.
- If no ENTRYPOINT is defined, CMD can also define the command to run.
- Useful for providing sensible defaults.

Dockerfile

```
CMD ["--help"]
```

This provides `--help` as the default argument (when no other argument is given).

How They Work Together

ENTRYPOINT defines the command, and **CMD** provides default arguments for that command.

Dockerfile

```
ENTRYPOINT ["/usr/bin/myapp"]  
CMD ["--config", "/etc/myapp.conf"]
```

What actually runs?

Run without arguments

```
docker run myimage
```

Runs: `/usr/bin/myapp --config /etc/myapp.conf`

Run with arguments

```
docker run myimage --verbose
```

Runs: `/usr/bin/myapp --verbose`

The provided arguments override the **CMD**.

If No ENTRYPOINT is Set

CMD becomes the command to run.

Dockerfile

```
CMD ["nginx", "-g", "daemon off;"]
```

```
docker run myimage
```

Runs: `nginx -g daemon off;`

What Happens if You Provide Multiple ENTRYPOINT or CMD Instructions?

Multiple ENTRYPOINT instructions



- **Not allowed.**
- If you specify more than one ENTRYPOINT in a Dockerfile, the **last one** will take effect (earlier ones are ignored).

Dockerfile

```
ENTRYPOINT ["/first"]  
ENTRYPOINT ["/second"] # this one wins
```

Result: Only `/second` is used.

Multiple CMD instructions



- **Allowed**, but only the **last one** takes effect.
- Previous CMD instructions are ignored.

Dockerfile

```
CMD ["--help"]  
CMD ["--version"] # this one wins
```

Result: Only `--version` is used.



Key Takeaways

- Use **ENTRYPOINT** for the main command (executable).
- Use **CMD** for default arguments.
- You can override **CMD** by specifying arguments in `docker run`.
- You can override **ENTRYPOINT** with `--entrypoint`.
- If multiple **ENTRYPOINT** or **CMD** instructions are defined, only the **last one** takes effect.

*Build images.
Run your way.*

docker exec

Run a command inside a running container

Get a shell, run a command, inspect, debug – without restarting the container!

What is docker exec?

docker exec allows you to run a command inside a running container. It does not start a new container – it executes the command inside an existing one.

How it works



Basic Syntax

```
docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

OPTIONS (optional) Flags like `-it`, `-u`, `-w`, etc.

CONTAINER Name or ID of the running container

COMMAND Command to run inside the container

ARG... (optional) Arguments for the command

Common Examples

1. Open an interactive shell (bash) # Get a bash shell inside the container

```
docker exec -it mycontainer bash
```

2. Run a single command # List files inside the container

```
docker exec mycontainer ls -l /app
```

3. Run a command with root user # Run as root user

```
docker exec -u root mycontainer whoami
```

4. Run a command in a specific working directory # Run command in /app directory

```
docker exec -w /app mycontainer pwd
```

Real-World Example

1. Start a container (in detached mode)

```
docker run -d --name web nginx
```

2. Exec into it and check the nginx version

```
docker exec web nginx -v
```

```
nginx version: nginx/1.25.3
```

Useful Options

<code>-i</code>	Keep STDIN open (interactive)
<code>-t</code>	Allocate a pseudo-TTY (terminal)
<code>-it</code>	Interactive terminal (commonly used)
<code>-u <user></code>	Run as a specific user
<code>-w <dir></code>	Set working directory inside container
<code>-e <env></code>	Set environment variables

Key Points

- ✓ Works only on running containers.
- ✓ Does not create a new container.
- ✓ Useful for debugging, troubleshooting, and running ad-hoc commands.
- ✓ Commonly used with `-it` to get an interactive shell.
- ✓ Supports running commands as different users, in different directories, and with environment variables.

docker exec – go inside, do what you need, and keep your container running!





Docker Prune

"Build. Test. Clean.
Keep it lean!"

Clean up unused Docker resources to free disk space

What is docker prune?

docker prune is a set of commands used to remove unused (dangling) Docker resources such as containers, images, networks, and build cache.

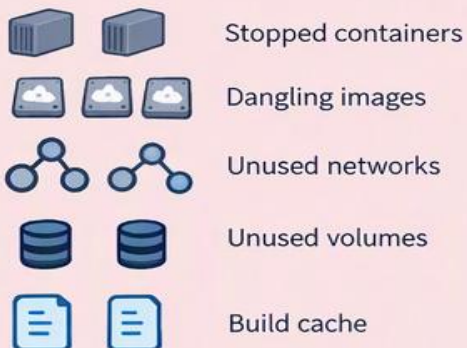
It helps you free up disk space and keep your Docker environment clean.



It only removes resources that are not in use. Running containers, images, networks, etc. are not affected.

Before and after

Before (cluttered)



Run
docker system prune
(or specific prune commands)

After (cleaner)



Unused resources removed
More disk space
A cleaner Docker environment



Only unused resources are removed. Your running containers and important data remain safe.

What can you prune?



Containers

Stopped containers



Images

Unused (dangling) images



Networks

Unused networks



Volumes

Unused volumes
(not removed by default)



Build cache

Unused build cache

Common docker prune commands

```
docker container prune
```

Remove stopped containers

```
docker image prune
```

Remove unused (dangling) images

```
docker network prune
```

Remove unused networks

```
docker volume prune
```

Remove unused volumes

```
docker builder prune
```

Remove unused build cache

```
docker system prune
```

Remove all unused resources (containers, images, networks, and build cache)

Optional flags

```
docker system prune -a
```

Also remove all unused images (not just dangling ones)

```
docker system prune --volumes
```

Include unused volumes

Things to keep in mind

- ✓ docker prune only removes unused resources.
- ✓ It will ask for confirmation. Use `-f` to skip the prompt.
- ✓ Use `docker system prune -a` with caution, as it removes all unused images (not just dangling ones).
- ✓ Volumes are **not** removed by default. Add `--volumes` if you also want to remove unused volumes.
- ✓ Regularly pruning can help save disk space, especially in development environments.



A cleaner Docker
today, a smoother tomorrow!



Docker Prune

Clean up unused Docker resources to free up disk space.

docker prune removes unused (dangling) Docker objects such as containers, images, networks, and volumes. It helps keep your system clean and saves disk space.

Before Prune

Docker has many unused resources (consuming disk space).



Containers
(stopped)

5

1.2 GB



Images
(dangling)

8

3.4 GB



Networks
(unused)

3

120 MB



Volumes
(unused)

4

2.1 GB



Build Cache
(unused)

6

1.8 GB



Run
docker prune
(or specific
prune commands)

After Prune

Only the resources in use remain. Disk space is reclaimed!



Containers
(stopped)

1

200 MB



Images
(dangling)

2

500 MB



Networks
(unused)

1

50 MB



Volumes
(unused)

1

100 MB



Build Cache
(unused)

0

0 MB



Common Docker Prune Commands

Command	What it removes	Example
<code>docker container prune</code>	Removes stopped containers	<code>docker container prune</code>
<code>docker image prune</code>	Removes dangling (untagged) images	<code>docker image prune</code>
<code>docker network prune</code>	Removes unused networks	<code>docker network prune</code>
<code>docker volume prune</code>	Removes unused volumes	<code>docker volume prune</code>
<code>docker builder prune</code>	Removes build cache	<code>docker builder prune</code>
<code>docker system prune</code>	Removes all unused resources (containers, images, networks, build cache)	<code>docker system prune</code>
<code>docker system prune -a</code>	Removes all unused resources (including unused images, not just dangling)	<code>docker system prune -a</code>
<code>docker system prune -a --volumes</code>	Removes all unused resources including volumes	<code>docker system prune -a --volumes</code>



Useful Options

`-a, --all`

Remove all unused images (not just dangling)

`--volumes`

Also remove unused volumes (use with system prune)

`-f, --force`

Skip confirmation prompt

`--filter`

Filter resources to prune (e.g. `--filter "until=24h"`)



Important

- Prune only removes resources that are NOT in use.
- Running containers, in-use images, networks, and volumes are not removed.
- Use `--volumes` with caution (it will delete unused volumes).
- There is no undo, so make sure you don't need the unused data.



When to Use?

- When disk space is running low.
- After development/testing to clean up old resources.
- As part of a regular cleanup routine (e.g. cron job).



A cleaner Docker environment = more space, better performance, and less clutter!

Build more. Worry less.

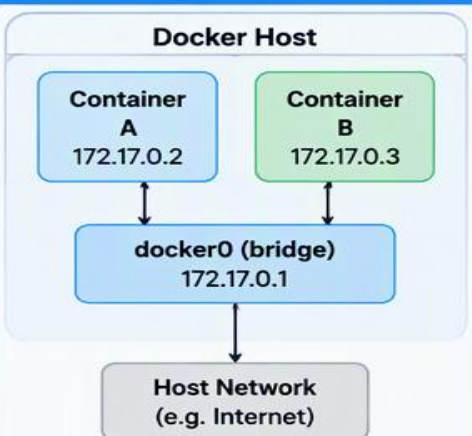


Docker Networking

Connect containers, enable communication, and integrate with the outside world.

Docker networking allows containers to communicate with each other, with the host, and with external networks using different network drivers.

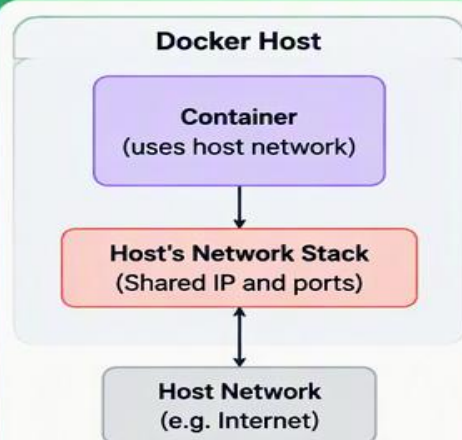
1. Bridge Network (Default)



- Default network for containers
- Containers get private IP addresses
- Containers can communicate with each other
- Outbound internet access via NAT
- Use for general purposes

```
# run a container (uses bridge by default)
docker run -d --name web nginx
```

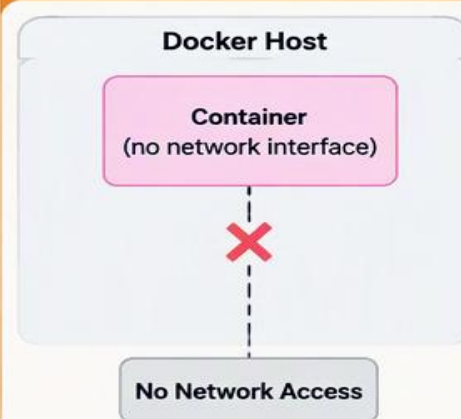
2. Host Network



- Container uses the host's network directly
- No network isolation
- Container shares the host's IP and ports
- Better performance (no NAT)
- Useful for high-performance apps

```
docker run -d --network host nginx
```

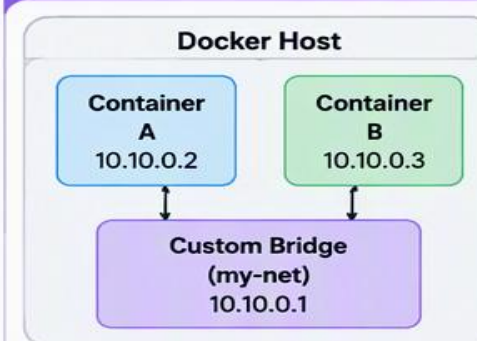
3. None Network



- Container has no network interfaces
- Cannot communicate with other containers, host, or internet
- Useful for highly isolated workloads that don't need networking

```
docker run -d --network none alpine
```

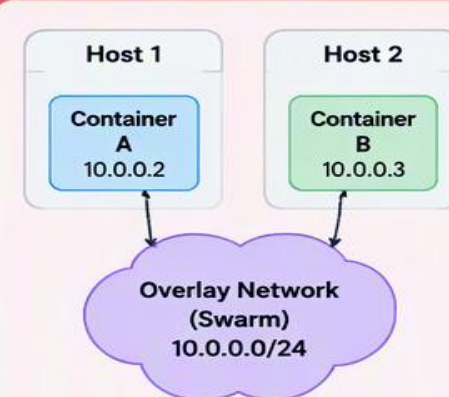
4. Custom Bridge Network



- User-defined bridge network
- Provides automatic DNS (container name resolution)
- Better isolation than default bridge
- Recommended for multi-container applications

```
docker network create my-net
docker run -d --network my-net \
  --name appl nginx
```

5. Overlay Network



- Connects containers across multiple Docker hosts
- Used with Docker Swarm (or other multi-host setups)
- Enables communication between containers on different hosts
- Useful for distributed applications

```
docker network create -d overlay
my-overlay
```



Common Docker Network Commands

<code>docker network ls</code>	<code># List networks</code>
<code>docker network create <name></code>	<code># Create a network</code>
<code>docker network inspect <name></code>	<code># Inspect a network</code>
<code>docker network rm <name></code>	<code># Remove a network</code>
<code>docker run --network <name> <image></code>	<code># Run a container on a network</code>



Key Takeaways

- ✓ **Bridge:** Default, good for most use cases on a single host.
- ✓ **Host:** Shares the host's network, no isolation.
- ✓ **None:** No networking, fully isolated.
- ✓ **Custom Bridge:** Better control and DNS, recommended for multi-container apps.
- ✓ **Overlay:** Enables communication across multiple hosts (Docker Swarm).

*Build
Ship
Run Anywhere*